



European IT Certification Curriculum Self-Learning Preparatory Materials

EITC/AI/MLP
Machine Learning with Python



This document constitutes European IT Certification curriculum self-learning preparatory material for the EITC/AI/MLP Machine Learning with Python programme.

This self-learning preparatory material covers requirements of the corresponding EITC certification programme examination. It is intended to facilitate certification programme's participant learning and preparation towards the EITC/AI/MLP Machine Learning with Python programme examination. The knowledge contained within the material is sufficient to pass the corresponding EITC certification examination in regard to relevant curriculum parts. The document specifies the knowledge and skills that participants of the EITC/AI/MLP Machine Learning with Python certification programme should have in order to attain the corresponding EITC certificate.

Disclaimer

This document has been automatically generated and published based on the most recent updates of the EITC/AI/MLP Machine Learning with Python certification programme curriculum as published on its relevant webpage, accessible at:

<https://eitca.org/certification/eitc-ai-mlp-machine-learning-with-python/>

As such, despite every effort to make it complete and corresponding with the current EITC curriculum it may contain inaccuracies and incomplete sections, subject to ongoing updates and corrections directly on the EITC webpage. No warranty is given by EITCI as a publisher in regard to completeness of the information contained within the document and neither shall EITCI be responsible or liable for any errors, omissions, inaccuracies, losses or damages whatsoever arising by virtue of such information or any instructions or advice contained within this publication. Changes in the document may be made by EITCI at its own discretion and at any time without notice, to maintain relevance of the self-learning material with the most current EITC curriculum. The self-learning preparatory material is provided by EITCI free of charge and does not constitute the paid certification service, the costs of which cover examination, certification and verification procedures, as well as related infrastructures.

EITC/AI/MLP MACHINE LEARNING WITH PYTHON DIDACTIC MATERIALS**LESSON: REGRESSION****TOPIC: INTRODUCTION TO REGRESSION****INTRODUCTION**

Artificial Intelligence - Machine Learning with Python - Regression - Introduction to Regression

Regression analysis is a fundamental statistical technique used in machine learning to model the relationship between a dependent variable and one or more independent variables. It is widely employed in various domains, including finance, economics, social sciences, and engineering. In this didactic material, we will explore the basics of regression analysis, focusing on the concept of linear regression and its implementation with Python.

Linear regression is a simple yet powerful technique used to predict a continuous dependent variable based on one or more independent variables. The relationship between the dependent variable and the independent variable(s) is assumed to be linear, which means that the change in the dependent variable can be explained by a linear combination of the independent variables. The goal of linear regression is to find the best-fitting line that minimizes the difference between the observed values and the predicted values.

To illustrate the concept of linear regression, let's consider a simple example. Suppose we want to predict the price of a house based on its size. We have a dataset that contains the sizes and prices of several houses. The size of the house would be the independent variable, also known as the predictor variable, while the price would be the dependent variable, also known as the target variable. By fitting a linear regression model to this dataset, we can estimate the relationship between the size and the price of a house.

In Python, we can use the scikit-learn library to perform linear regression. Scikit-learn is a powerful machine learning library that provides a wide range of tools and algorithms for regression analysis. To use scikit-learn, we first need to import the necessary modules and load the dataset into our Python environment. Once the dataset is loaded, we can split it into training and testing sets to evaluate the performance of our regression model.

The next step is to create an instance of the linear regression model and fit it to the training data. This involves finding the optimal values for the coefficients of the linear equation that best represents the relationship between the independent and dependent variables. The coefficients can be interpreted as the weights assigned to each independent variable in the linear equation. By adjusting these weights, the model can make accurate predictions based on the input data.

After fitting the model, we can use it to make predictions on the testing data. The performance of the model can be evaluated using various metrics, such as mean squared error (MSE) or R-squared. The MSE measures the average squared difference between the predicted and actual values, while R-squared indicates the proportion of the variance in the dependent variable that can be explained by the independent variables.

In addition to linear regression, there are other types of regression techniques, such as polynomial regression, ridge regression, and lasso regression. These techniques allow for more complex relationships between the independent and dependent variables. Polynomial regression, for example, can capture nonlinear relationships by introducing polynomial terms into the linear equation. Ridge regression and lasso regression, on the other hand, are regularization techniques that help prevent overfitting by adding a penalty term to the loss function.

Regression analysis is a fundamental technique in machine learning that allows us to model and predict the relationship between a dependent variable and one or more independent variables. Linear regression is a simple yet powerful method for predicting continuous variables based on linear relationships. With the help of Python and libraries like scikit-learn, we can easily implement and evaluate regression models. By understanding regression analysis, we can gain valuable insights and make accurate predictions in various domains.

DETAILED DIDACTIC MATERIAL

Regression is a technique used in machine learning to find the best fit line for continuous data. It involves modeling the relationship between features (attributes) and labels (continuous data) to predict future outcomes. In this example, we will use regression to analyze stock prices.

To get started, we need to install the necessary libraries. Open up the terminal or command prompt and install scikit-learn, Pandas, and Quandl by using the pip install command. Once installed, we can proceed with the example.

In regression, the equation of a straight line is used to model the data. The equation, $y = mx + b$, represents the dependent variable (y) as a function of the independent variable (x), with m as the slope and b as the y-intercept. The goal of regression is to determine the values of m and b that best fit the data.

In this case, we will be using the Quandl library to obtain stock price data. By specifying the ticker symbol, we can retrieve the dataset from Quandl. The dataset contains various features such as open, high, low, close, volume, and adjusted prices. These features represent different attributes of the stock.

When working with features, it is important to consider their relevance and meaningfulness to the data. Not all features may contribute significantly to the pattern recognition process. In this example, we will focus on the open, high, low, close, and adjusted prices, as they are closely related and provide meaningful data for our analysis.

Adjusted prices account for stock splits, where the number of shares and their prices are adjusted to maintain consistency. This ensures that the stock price does not appear to have changed drastically due to a split. We will be using the adjusted prices in our regression analysis.

It is worth noting that in regression, we do not consider the relationships between different features. However, in other machine learning algorithms such as deep learning, relationships between attributes can be explored. For regression, simplicity is key, and we aim to use meaningful features that have a direct impact on the data.

By simplifying our data and selecting relevant features, we can improve the accuracy and efficiency of our regression model. In the next steps, we will explore and analyze the dataset to further understand the relationship between the features and labels.

In this didactic material, we will introduce the concept of regression in the context of machine learning with Python. Regression is a supervised learning technique used to predict continuous numerical values based on input features. We will explore how to select relevant features and create a dataframe for regression analysis.

When working with machine learning classifiers, it is important to avoid including useless features as they can cause trouble. In supervised learning, especially with simpler classifiers, useless features can negatively impact the accuracy of predictions. Therefore, it is important to carefully choose the features that will be used in the analysis.

To begin, we will create a dataframe by selecting specific columns from an existing dataframe. The columns we will include are 'adjusted', 'open', 'high', 'low', 'close', and 'volume'. By selecting these columns, we are essentially recreating the dataframe to only include the relevant information needed for our analysis.

Some of these columns may seem relatively worthless at first glance, but they still hold valuable relationships. For example, the margin between the 'high' and 'low' prices can provide insights into the volatility of a given day. Similarly, the relationship between the 'open' and 'close' prices can indicate whether the price went up or down and by how much. These relationships can be valuable in predicting future outcomes.

However, a simple linear regression model will not automatically identify these relationships. It will only work with the features provided to it. Therefore, it is necessary to define these special relationships explicitly and use them as features in the regression analysis. By doing so, we can avoid redundant features that do not provide additional useful information.

Let's start by calculating the 'high minus low percent', which represents the percent volatility. We will define a new column called 'HL_percent', which is calculated as the difference between the 'high' and 'low' prices divided by the 'low' price, multiplied by 100. This calculation is performed on a per-row basis.

Next, we will calculate the 'percent_change', which represents the daily percent movement. This is calculated similarly to the 'HL_percent', but using the 'adjusted close' and 'adjusted open' prices instead. The 'percent_change' is calculated as the difference between the 'adjusted close' and 'adjusted open' prices divided by the 'adjusted open' price, multiplied by 100.

Once we have calculated these new columns, we will define a new dataframe that only includes the columns we are interested in. In our case, the columns we care about are 'adjusted close', 'high low percent', 'percent change', and 'volume'. These columns provide us with the necessary information for our regression analysis.

Finally, we will print the first few rows of the dataframe to ensure that everything has been set up correctly. This step allows us to verify that the dataframe includes the desired columns and that the data is correctly formatted.

In the next tutorial, we will delve deeper into the concept of labels and features. We will explore whether the 'adjusted close' column will be used as a feature or as a label. This decision will depend on the specific goals of our analysis and the predictions we want to make.

If you have any questions or comments, please leave them below. Thank you for watching, and we appreciate your support and subscriptions. Stay tuned for the next tutorial, where we will continue our exploration of regression analysis with real data.

EITC/AI/MLP MACHINE LEARNING WITH PYTHON - REGRESSION - INTRODUCTION TO REGRESSION - REVIEW QUESTIONS:**WHAT IS REGRESSION IN THE CONTEXT OF MACHINE LEARNING, AND HOW IS IT USED TO PREDICT FUTURE OUTCOMES?**

Regression is a fundamental concept in the field of machine learning, specifically in the context of predictive modeling. It is a statistical approach that aims to establish a relationship between a dependent variable and one or more independent variables. This relationship is then utilized to predict future outcomes or estimate the value of the dependent variable based on the given independent variables.

In machine learning, regression can be categorized into two main types: simple regression and multiple regression. Simple regression involves a single independent variable, whereas multiple regression involves multiple independent variables. Both types rely on a set of training data to learn the relationship between the variables and build a predictive model.

The goal of regression is to find the best-fitting line or curve that represents the relationship between the independent and dependent variables. This line or curve is determined by minimizing the sum of the squared differences between the predicted and actual values of the dependent variable. This approach is known as the least squares method.

To predict future outcomes using regression, the trained model is applied to new data where the values of the independent variables are known. The model then calculates the predicted value of the dependent variable based on the learned relationship. This prediction can be a single point estimate or a range of values, depending on the specific regression technique used.

For example, let's consider a simple regression problem where we want to predict the price of a house based on its size. We have a dataset containing the sizes and prices of several houses. By applying regression, we can find the best-fitting line that represents the relationship between the size (independent variable) and the price (dependent variable). Once the model is trained, we can use it to predict the price of a new house given its size.

Regression has various applications across different domains. It is commonly used in finance to predict stock prices, in healthcare to estimate patient outcomes, in marketing to forecast sales, and in many other fields where predicting future outcomes based on historical data is essential.

Regression is a powerful technique in machine learning that enables us to predict future outcomes or estimate the value of a dependent variable based on one or more independent variables. By establishing a relationship between the variables through training data, regression models can provide valuable insights and predictions in various domains.

WHAT ARE THE NECESSARY LIBRARIES THAT NEED TO BE INSTALLED TO PERFORM REGRESSION ANALYSIS IN PYTHON?

To perform regression analysis in Python, there are several necessary libraries that need to be installed. These libraries provide the essential tools and functions required for regression analysis tasks. In this answer, we will explore the key libraries used in Python for regression analysis and discuss their functionalities and applications.

1. NumPy:

NumPy is a fundamental library for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently. NumPy is commonly used to handle data preprocessing and manipulation tasks in regression analysis.

Example:

1.	<code>import numpy as np</code>
2.	<code># Create a NumPy array</code>
3.	<code>data = np.array([1, 2, 3, 4, 5])</code>
4.	<code># Calculate the mean of the array</code>
5.	<code>mean = np.mean(data)</code>
6.	<code>print("Mean:", mean)</code>

2. pandas:

pandas is a powerful data manipulation library that provides data structures like DataFrames, which allow for easy handling and analysis of structured data. It offers various functionalities for data preprocessing, cleaning, and transformation, making it a valuable tool for regression analysis.

Example:

1.	<code>import pandas as pd</code>
2.	<code># Create a pandas DataFrame</code>
3.	<code>data = pd.DataFrame({'x': [1, 2, 3, 4, 5], 'y': [2, 4, 5, 4, 6]})</code>
4.	<code># Calculate the correlation between two columns</code>
5.	<code>correlation = data['x'].corr(data['y'])</code>
6.	<code>print("Correlation:", correlation)</code>

3. scikit-learn:

scikit-learn is a widely used machine learning library in Python. It provides a comprehensive set of tools for regression analysis, including various regression algorithms, evaluation metrics, and data preprocessing techniques. scikit-learn simplifies the implementation of regression models and allows for easy comparison and selection of different algorithms.

Example:

1.	<code>from sklearn.linear_model import LinearRegression</code>
2.	<code>from sklearn.metrics import mean_squared_error</code>
3.	<code>from sklearn.model_selection import train_test_split</code>
4.	<code># Load the dataset</code>
5.	<code>data = pd.read_csv('data.csv')</code>
6.	<code># Split the data into features and target variable</code>
7.	<code>X = data[['x1', 'x2', 'x3']]</code>
8.	<code>y = data['y']</code>
9.	<code># Split the data into training and testing sets</code>
10.	<code>X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)</code>
11.	<code># Create a linear regression model</code>
12.	<code>model = LinearRegression()</code>
13.	<code># Fit the model to the training data</code>
14.	<code>model.fit(X_train, y_train)</code>
15.	<code># Predict the target variable for the test data</code>
16.	<code>y_pred = model.predict(X_test)</code>
17.	<code># Calculate the mean squared error</code>
18.	<code>mse = mean_squared_error(y_test, y_pred)</code>
19.	<code>print("Mean Squared Error:", mse)</code>

4. matplotlib:

matplotlib is a plotting library that allows for the creation of various types of visualizations, such as line plots, scatter plots, and histograms. It is often used in regression analysis to visualize the relationship between variables and the performance of regression models.

Example:

1.	<code>import matplotlib.pyplot as plt</code>
2.	<code># Create scatter plot of the data</code>
3.	<code>plt.scatter(data['x'], data['y'])</code>
4.	<code>plt.xlabel('x')</code>
5.	<code>plt.ylabel('y')</code>
6.	<code>plt.title('Scatter Plot')</code>
7.	<code>plt.show()</code>

These libraries, NumPy, pandas, scikit-learn, and matplotlib, are essential for performing regression analysis in Python. They offer a wide range of functionalities for data manipulation, model building, evaluation, and visualization. By leveraging the capabilities of these libraries, researchers and practitioners can effectively analyze and model relationships between variables in regression tasks.

WHAT IS THE EQUATION USED TO MODEL THE RELATIONSHIP BETWEEN FEATURES AND LABELS IN REGRESSION?

The equation used to model the relationship between features and labels in regression is known as the regression equation or the hypothesis function. In regression, we aim to predict a continuous output variable (label) based on one or more input variables (features). The regression equation allows us to express this relationship mathematically.

In its simplest form, the regression equation for a single feature is expressed as:

$$y = mx + b$$

where:

- y represents the predicted output variable or label,
- x represents the input feature,
- m represents the slope of the regression line, which determines the direction and steepness of the relationship between x and y,
- b represents the y-intercept, which is the value of y when x is equal to zero.

For example, let's say we want to predict the price of a house (y) based on its size in square feet (x). We can use the regression equation to model this relationship:

$$\text{price} = \text{slope} * \text{size} + \text{intercept}$$

The slope (m) represents how much the price changes for every unit increase in size, and the intercept (b) represents the price when the size is zero (which may not be meaningful in this context).

In multiple linear regression, where we have more than one input feature, the regression equation becomes:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

where:

- y represents the predicted output variable or label,
- $x_1, x_2, \dots, x_n$ represent the input features,

- b_0 represents the y-intercept,
- $b_1, b_2, \dots, b_n$ represent the coefficients that determine the relationship between each input feature and the output variable.

For instance, if we want to predict the price of a house (y) based on its size (x_1) and number of bedrooms (x_2), the regression equation would be:

$$\text{price} = \text{intercept} + \text{coefficient1} * \text{size} + \text{coefficient2} * \text{bedrooms}$$

The intercept (b_0) represents the price when both size and number of bedrooms are zero, while the coefficients (b_1, b_2) indicate how much the price changes for each unit increase in size or number of bedrooms.

It is important to note that the regression equation is typically learned from a given dataset using various regression algorithms, such as ordinary least squares (OLS), gradient descent, or support vector regression (SVR). These algorithms estimate the coefficients in the equation by minimizing the difference between the predicted values and the actual values of the output variable.

The equation used to model the relationship between features and labels in regression is a fundamental tool in machine learning. It allows us to express the relationship between input variables and the output variable mathematically, enabling us to make predictions based on new input data.

WHY IS IT IMPORTANT TO CONSIDER THE RELEVANCE AND MEANINGFULNESS OF FEATURES WHEN WORKING WITH REGRESSION ANALYSIS?

When working with regression analysis in the field of artificial intelligence and machine learning, it is important to consider the relevance and meaningfulness of the features used. This is important because the quality of the features directly impacts the accuracy and interpretability of the regression model. In this answer, we will explore the reasons why feature relevance and meaningfulness are essential in regression analysis, providing a comprehensive explanation of their didactic value based on factual knowledge.

Firstly, relevance refers to the degree to which a feature is related to the target variable or outcome of interest. In regression analysis, the goal is to build a model that accurately predicts the target variable based on the input features. If irrelevant features are included in the model, they can introduce noise and hinder the model's performance. Irrelevant features may not contribute any meaningful information to the model, leading to overfitting and poor generalization. Overfitting occurs when the model learns the noise or random fluctuations in the training data instead of the underlying patterns, resulting in low predictive performance on unseen data.

For example, suppose we are building a regression model to predict house prices based on various features such as the number of bedrooms, square footage, and location. Including an irrelevant feature like the color of the front door, which has no real impact on house prices, would introduce noise and potentially degrade the model's accuracy. By considering the relevance of features, we can focus on those that have a significant impact on the target variable, leading to a more accurate and interpretable model.

Secondly, meaningfulness refers to the practical significance or interpretability of the features. In many real-world applications, it is important to understand the relationship between the input features and the target variable. Meaningful features provide insights into the underlying mechanisms or causal relationships in the data, enabling us to make informed decisions or draw meaningful conclusions.

For instance, in a medical study aiming to predict the risk of heart disease based on various patient characteristics, meaningful features such as blood pressure, cholesterol levels, and smoking status would provide valuable insights into the factors contributing to the disease. On the other hand, including irrelevant or nonsensical features like the patient's favorite color or shoe size would not contribute to our understanding of the problem and could potentially lead to misleading results.

Moreover, meaningful features can help in feature selection and dimensionality reduction. Feature selection techniques aim to identify the most relevant features that have the most impact on the target variable while discarding irrelevant or redundant ones. By considering the meaningfulness of features, we can prioritize those

that provide the most valuable information, leading to simpler and more interpretable models. This is particularly important when dealing with high-dimensional data, where the number of features is large compared to the number of samples.

Considering the relevance and meaningfulness of features is important when working with regression analysis in the field of artificial intelligence and machine learning. Relevant features contribute to accurate predictions by providing meaningful information, while meaningful features enhance our understanding of the underlying relationships in the data. By carefully selecting and interpreting features, we can build models that are more accurate, interpretable, and useful in real-world applications.

WHAT ARE ADJUSTED PRICES IN THE CONTEXT OF STOCK ANALYSIS, AND WHY ARE THEY USED IN REGRESSION ANALYSIS?

Adjusted prices, in the context of stock analysis, refer to the prices of stocks that have been modified to account for certain factors, such as stock splits, dividends, or other corporate actions. These adjustments are made to ensure that the prices accurately reflect the underlying value of the stock and provide a more meaningful representation for analysis and modeling purposes.

One common reason for using adjusted prices in regression analysis is to account for the effects of stock splits. A stock split occurs when a company decides to divide its existing shares into multiple shares. For example, a 2-for-1 stock split would result in each existing share being divided into two shares. As a result of the split, the price of each share is halved. However, the total value of the investment remains the same.

When conducting regression analysis, it is important to consider the impact of stock splits on the historical price data. If the raw price data is used without any adjustments, the analysis may be skewed and inaccurate. By using adjusted prices, the effects of stock splits are eliminated, allowing for a more accurate analysis of the relationship between variables.

Another reason for using adjusted prices in regression analysis is to account for the effects of dividends. Dividends are payments made by a company to its shareholders as a distribution of profits. When a dividend is paid, the stock price typically decreases by the amount of the dividend. This decrease in price can have an impact on the analysis if the raw price data is used.

By using adjusted prices, the effects of dividends are taken into account, ensuring that the analysis is not biased by these payments. This is particularly important when analyzing long-term trends or conducting predictive modeling, as the impact of dividends can be significant over time.

In addition to stock splits and dividends, there may be other corporate actions or events that can impact the price of a stock. These can include mergers, acquisitions, spin-offs, or stock buybacks. Adjusted prices are used to account for these events and provide a more accurate representation of the underlying value of the stock.

To calculate adjusted prices, various methods can be used, depending on the specific corporate actions and events. For example, when adjusting for stock splits, the historical prices are divided by the split ratio to reflect the new number of shares. When adjusting for dividends, the historical prices are decreased by the amount of the dividend.

Adjusted prices in stock analysis refer to prices that have been modified to account for stock splits, dividends, and other corporate actions. These adjustments are important in regression analysis to ensure that the analysis is not biased by these factors. By using adjusted prices, the effects of stock splits and dividends are eliminated, providing a more accurate representation of the underlying value of the stock.